

🌟 DAY - 15 : Javascript Arrays 🚀 🌟

JS JAVASCRIPT

Level Beginner

📖 From Basics to Advanced 📖

📌 Introduction

Arrays are one of the most fundamental and powerful data structures in JavaScript. They allow you to store multiple values in a single variable, which makes organizing and manipulating data much easier and more efficient.

In this lesson, we'll explore arrays in depth, from creating them to manipulating them, and learn about modern JavaScript features like destructuring, rest parameters, and the spread operator.

🔍 What is an Array in JavaScript?

An array is an ordered collection of values that can be accessed by their position (index). In JavaScript:

- Arrays can store any type of data (numbers, strings, objects, other arrays, etc.)
- Array indices start at 0 (zero-based indexing)
- Arrays are dynamic - they can grow or shrink as needed
- Arrays are objects with special behaviors and methods

```
// Array examples
const numbers = [1, 2, 3, 4, 5];           // 📦 Array of numbers
const fruits = ['apple', 'banana', 'orange']; // 🍎 Array of strings
const mixedArray = [100, true, 'taPaScript', {}]; // 🗂 Mixed types
const nestedArray = [1, 2, [3, 4, [5, 6]]]; // 📦 Nested arrays
```

💡 **Key Point:** JavaScript arrays are special objects with numeric keys and a `length` property, which makes them different from regular objects.

✅ Knowledge Check

★ **Question 1:** What is the starting index of a JavaScript array?

★ **Question 2:** Can JavaScript arrays store different data types in the same array?

How to Create an Array in JavaScript?

There are several ways to create arrays in JavaScript:

1 Array Literal (Most Common)

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];
```

2 Using the `Array()` Constructor

```
const anotherSalad = new Array("🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑");
```

⚠️ **Warning:** Be careful with the `new Array()` constructor! If you pass a single number, it creates an array with that many empty slots:

```
const five = new Array(5);    // Creates [<5 empty slots>]
const two = new Array(1, 2);  // Creates [1, 2]
```

3 `Array.of()` Method

```
const numbers = Array.of(1, 2, 3, 4, 5); // Creates [1, 2, 3, 4, 5]
const singleNumber = Array.of(5);        // Creates [5] (not an array of 5 empty slots)
```

4 `Array.from()` Method

Converts array-like or iterable objects into arrays:

```
const arrayFromString = Array.from("hello"); // Creates ["h", "e", "l", "l", "o"]
```

✅ Knowledge Check

 **Question 1:** What's the difference between `new Array(5)` and `Array.of(5)`?

 **Question 2:** Write code to create an array with the values 10, 20, and 30 using both array literal and constructor syntax.

How to Get/Access Elements from an Array in JS?

You can access array elements using square bracket notation with the index:

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

console.log(salad[0]);    // "🍎" (first element)
console.log(salad[2]);    // "🥬" (third element)
console.log(salad[5]);    // "🥕" (sixth element)
```

You can also loop through arrays to access each element:

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Using for loop
for (let i = 0; i < salad.length; i++) {
  console.log(`Element at index ${i} is ${salad[i]}`);
}

// Using for...of loop (ES6)
for (const vegetable of salad) {
  console.log(vegetable);
}

// Using forEach method
salad.forEach((vegetable, index) => {
  console.log(`Element at index ${index} is ${vegetable}`);
});
```

💡 **Key Point:** If you try to access an index that doesn't exist, JavaScript will return `undefined` rather than throwing an error.

✅ Knowledge Check

❌ **Question 1:** What is returned if you try to access an array element at an index that doesn't exist?

❌ **Question 2:** Write code to access the last element of an array without knowing its length in advance.

+ How to Add Elements to an Array in JS?

Adding Elements to the End: `push()`

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];
```

```
// Using push() method
const newLength = salad.push("🥜"); // Adds "🥜" to the end
console.log(newLength); // 8 (new length of the array)
console.log(salad);      // ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑", "🥜"]
```

Adding Elements to the Beginning: unshift()

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Using unshift() method
const newLength = salad.unshift("🥜"); // Adds "🥜" to the beginning
console.log(newLength); // 8 (new length of the array)
console.log(salad);      // ["🥜", "🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"]
```

Adding Elements at a Specific Position: splice()

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Using splice() method
// syntax: array.splice(startIndex, deleteCount, item1, item2, ...)
salad.splice(2, 0, "🥜"); // Insert "🥜" at index 2, delete 0 elements
console.log(salad);      // ["🍎", "🍄", "🥜", "🥬", "🥒", "🌽", "🥕", "🥑"]
```

💡 **Key Point:** `push()` and `unshift()` both return the new length of the array after the operation.

✅ Knowledge Check

🌱 **Question 1:** What methods can you use to add elements to an array?

🌱 **Question 2:** Which method would you use to insert an element in the middle of an array?

— How to Remove Elements from an Array in JS?

Removing Elements from the End: pop()

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Using pop() method
const removedItem = salad.pop(); // Removes the last element and returns it
console.log(removedItem);        // "🥑"
console.log(salad);              // ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕"]
```

Removing Elements from the Beginning: `shift()`

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Using shift() method
const removedItem = salad.shift(); // Removes the first element and returns it
console.log(removedItem);          // "🍎"
console.log(salad);                // ["🍄", "🥬", "🥒", "🌽", "🥕", "🥑"]
```

Removing Elements at a Specific Position: `splice()`

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Using splice() method
// syntax: array.splice(startIndex, deleteCount)
const removedItems = salad.splice(2, 2); // Remove 2 elements starting at index 2
console.log(removedItems);               // ["🥬", "🥒"]
console.log(salad);                     // ["🍎", "🍄", "🌽", "🥕", "🥑"]
```

💡 **Key Point:** `pop()` and `shift()` both return the removed element, while `splice()` returns an array of removed elements.

✅ Knowledge Check

- 🔁 Question 1: What is the difference between `pop()` and `shift()`?
- 🔁 Question 2: What do the methods `pop()` and `shift()` return when called on an array?

📋 How to Copy and Clone an Array in JS?

There are several ways to copy arrays in JavaScript:

1. Using `slice()` Method

```
const salad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];
const saladCopy = salad.slice(); // Creates a shallow copy of the array

console.log(salad === saladCopy); // false (different array references)
```

NOTE: Since `push()`, `unshift()`, `pop()` and `shift()` all these methods bring changes to the original array meaning that it `mutate` the source array but the `slice()` method does not mutate the source array meaning that it does not modify the original array rather creates a new copy of array. It deals in `immutable` way.

2. Using Spread Operator (ES6)

```
const salad = ["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];
const saladCopy = [...salad]; // Creates a shallow copy using spread operator

console.log(saladCopy); // ["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"]
console.log(salad === saladCopy); // false (different array references)
```

3. Using Array.from() Method

```
const salad = ["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];
const saladCopy = Array.from(salad); // Creates a shallow copy

console.log(saladCopy); // ["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"]
```

4. Using Array.concat() Method

```
const salad = ["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];
const saladCopy = [].concat(salad); // Creates a shallow copy

console.log(saladCopy); // ["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"]
```

💡 **Key Point:** All the methods above create shallow copies, meaning that nested arrays or objects still share references. For deep copying complex arrays, you might need to use methods like `JSON.parse(JSON.stringify())` or a library like Lodash.

✅ Knowledge Check

🔍 **Question 1:** What's the difference between a shallow copy and a deep copy?

🔍 **Question 2:** Which method would you use to create a copy of an array that includes only the first three elements?

🌟 How to Determine if a Value is an Array in JS?

JavaScript provides the `Array.isArray()` method to check if a value is an array:

```
Array.isArray(["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"]); // true
Array.isArray("🍅"); // false (string)
Array.isArray({ tomato: "🍅" }); // false (object)
Array.isArray([]); // true (empty array)
```

```
const arr = [1, 2, 3, 4];
Array.isArray(arr);           // true
```

💡 **Key Point:** The `typeof` operator doesn't help much with arrays because it returns "object" for arrays. Always use `Array.isArray()` to check specifically for arrays.

✅ Knowledge Check

💡 **Question 1:** Why doesn't `typeof arr === 'array'` work to check if a value is an array?

💡 **Question 2:** What does `typeof []` return in JavaScript?

🧩 Array Destructuring in JavaScript

Array destructuring is a convenient way to extract values from arrays and assign them to variables.

📌 Basic Destructuring

```
const salad = ["🍅", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Without destructuring 🚫
const tomato = salad[0];
const mushroom = salad[1];
const carrot = salad[5];

// With destructuring ✅
const [tomato, mushroom, , , , carrot] = salad;
console.log(tomato, mushroom, carrot); // "🍅" "🍄" "🥕"
```

📌 How to Assign a Default Value to a Variable?

You can provide default values in case the array doesn't have a value at that position:

```
const [tomato, mushroom = "🍄"] = ["🍅"];
console.log(tomato);    // "🍅"
console.log(mushroom);  // "🍄" (default value used) ⭐
```

📌 How to Skip a Value in an Array?

You can use commas to skip elements in the array:

```
const [tomato, , carrot] = ["🍅", "🍄", "🥕"];
console.log(tomato);    // "🍅"
```

```
console.log(carrot); // "🥕" (skipped "🍄") ▶
```

📌 Nested Array Destructuring in JS

You can destructure nested arrays too:

```
const fruits = ["🍏", "🍓", "🍌", "🍉", ["🍎", "🍄", "🥕"]];

// Without destructuring 🚫
const veg = fruits[4]; // ["🍎", "🍄", "🥕"]
const carrot = veg[2]; // "🥕"
const directCarrot = fruits[4][2]; // "🥕"

// With destructuring ✅
const [, , , , [, , carrot]] = ["🍏", "🍓", "🍌", "🍉", ["🍎", "🍄", "🥕"]];
console.log(carrot); // "🥕"
```

✅ Knowledge Check

🧩 Question 1: What happens if you try to destructure more variables than there are elements in the array?

🧩 Question 2: Write code to destructure the first and last elements of an array of unknown length.

💡 How to Use the Rest Parameter in JS?

The rest parameter (...) collects multiple elements into an array.

...rest --> at left side(variable naming side) becomes rest parameter.

```
const [tomato, mushroom, ...rest] = ["🍆", "🍄", "🍌", "🥒", "🍌", "🥕", "🥑"];
console.log(tomato); // "🍆"
console.log(mushroom); // "🍄"
console.log(rest); // ["🍌", "🥒", "🍌", "🥕", "🥑"]
```

💡 Key Point: The rest parameter must be the last parameter in the destructuring pattern.

✅ Knowledge Check

🧩 Question 1: What happens if you use the rest parameter in the middle of a destructuring pattern?

 **Question 2:** Can you use more than one rest parameter in a single destructuring?

How to Use the Spread Operator in JS?

The spread operator (`...`) expands an array into individual elements.

We can create a clone or copy of the array using `spread operator` .

`...arrayname` ---> at right side(value of array side) becomes spread operator`

```
const mySalad = ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"];

// Copying an array
const mySaladCopy = [...mySalad];
console.log(mySaladCopy); // ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑"]
console.log(mySalad === mySaladCopy); // false (different array references)

// Concatenating arrays
const moreSalad = [...mySalad, "🍌", "🍌"];
console.log(moreSalad); // ["🍎", "🍄", "🥬", "🥒", "🌽", "🥕", "🥑", "🍌", "🍌"]

// Passing array elements as function arguments
function makeSalad(a, b, c) {
  return `Salad with ${a}, ${b}, and ${c}`;
}
const ingredients = ["🍎", "🍄", "🥬"];
console.log(makeSalad(...ingredients)); // "Salad with 🍎, 🍄, and 🥬"
```

 **Key Point:** The spread operator is great for creating copies of arrays, combining arrays, and converting array-like objects to arrays.

Knowledge Check

 **Question 1:** What's the difference between the rest parameter and the spread operator?

 **Question 2:** How can you use the spread operator to create a new array with one item added at the beginning?

Destructuring Use Cases in JavaScript

How to Swap Values with Destructuring?

```
let first = "😞";
let second = "😄";

// Swap values using destructuring ✨
[first, second] = [second, first];

console.log(first); // "😄"
console.log(second); // "😞"
```

How to Merge Arrays?

```
const emotions = ["😄", "😞"];
const veggies = ["🥦", "🥒", "🌽", "🥕"];

// Merge arrays using spread operator ✨
const emotionalVeggies = [...emotions, ...veggies];
console.log(emotionalVeggies); // ["😄", "😞", "🥦", "🥒", "🌽", "🥕"]
```

✅ Knowledge Check

-  **Question 1:** Without using a temporary variable or destructuring, how would you swap two values?
-  **Question 2:** How would you merge three arrays into one using the spread operator?

The length property

The `length` property of an array returns the number of elements in the array:

```
const arr1 = [11, 21, 73];
console.log(arr1.length); // 3

const arr2 = new Array(7);
console.log(arr2.length); // 7 (an array with 7 empty slots)
```

You can also modify the `length` property to truncate or expand an array:

```
const arr = [1, 2, 3, 4, 5];
console.log(arr.length); // 5

// Truncate array
arr.length = 3;
console.log(arr); // [1, 2, 3]
```

```
// Expand array (adds empty slots)
arr.length = 6;
console.log(arr); // [1, 2, 3, empty × 3]
```

💡 **Key Point:** The maximum length of an array in JavaScript is $2^{32} - 1$ (4,294,967,295) elements.

✅ Knowledge Check

🔪 **Question 1:** What happens when you set the length property of an array to a value less than its current length?

🔪 **Question 2:** What happens when you set the length property of an array to a value greater than its current length?

🛠️ Practice Tasks

Test your understanding of JavaScript Arrays with these practice tasks:

1. **Create an array** of your five favorite foods.

2. **Access elements:**

- Get the first and last elements of your array.
- Try accessing an element at an index that doesn't exist. What happens?

3. **Modify your array:**

- Add a new food to the beginning of your array.
- Add a new food to the end of your array.
- Remove the first food from your array.
- Remove the last food from your array.

4. **Create a function** that takes your array and returns a new array with all elements in reverse order.

5. **Create a function** that merges two arrays and removes any duplicate elements.

6. **Array of objects:**

- Create an array of objects where each object represents a person with properties for name, age, and city.
- Write code to find all people who are older than 30.

- Sort the array by age in descending order.

7. Nested arrays:

- Create a 3x3 grid represented as a nested array.
- Write a function to get the element at a specific row and column.
- Write a function to calculate the sum of all elements in the grid.

8. Destructuring challenge:

- Given the array `const data = [1, [2, 3], 4, [5, [6, 7]]]`, use destructuring to get the values 1, 3, and 7 in separate variables in one line.

9. Spread operator challenge:

- Given two arrays `const arr1 = [1, 2, 3]` and `const arr2 = [4, 5, 6]`, create a new array that has the elements in the following order: `[4, 5, 6, 1, 2, 3, 0]`.

10. Real-world application:

- Create an array of objects representing tasks in a to-do list.
- Write functions to add a task, mark a task as completed, and delete a task.
- Write a function to filter tasks by their completion status.
- Use array methods to sort tasks by due date.

JavaScript Array Methods & Immutability

JavaScript Array Methods

Array methods are built-in functions that allow you to perform operations on arrays. They make working with arrays much more efficient and readable.

The `concat()` Method

The `concat()` method merges two or more arrays and returns a new array without modifying the original arrays.

Syntax:

```
array.concat(array1, array2, ..., arrayX)
```

Example:

```
const fruits = ['apple', 'banana'];
const vegetables = ['carrot', 'tomato'];
const combined = fruits.concat(vegetables);

console.log(combined); // ['apple', 'banana', 'carrot', 'tomato']
console.log(fruits);   // ['apple', 'banana'] (unchanged)
```

You can concatenate multiple arrays at once:

```
const meats = ['chicken', 'beef'];
const groceries = fruits.concat(vegetables, meats);

console.log(groceries); // ['apple', 'banana', 'carrot', 'tomato', 'chicken', 'beef']
```

💡 **Knowledge Check:** What happens to the original arrays when you use `concat()` ?

► Click to see answer

The join() Method

The `join()` method creates and returns a new string by concatenating all elements in an array, separated by a specified separator.

Syntax:

```
array.join(separator)
```

Example:

```
const fruits = ['apple', 'banana', 'orange'];

// Default separator is comma (,)
console.log(fruits.join()); // 'apple,banana,orange'

// Custom separators
console.log(fruits.join(' - ')); // 'apple - banana - orange'
console.log(fruits.join(''));   // 'applebananaorange'
console.log(fruits.join(' '));  // 'apple banana orange'
```

💡 **Knowledge Check:** What's the default separator used when no argument is provided to `join()` ?

► Click to see answer

The fill() Method

The `fill()` method changes all elements in an array to a static value and returns the modified array.

Syntax:

```
array.fill(value, start, end)
```

- `value` : The value to fill the array with
- `start` : (Optional) Start index, default is 0
- `end` : (Optional) End index (exclusive), default is array length

Example:

```
const numbers = [1, 2, 3, 4, 5];

// Fill entire array with value 0
console.log(numbers.fill(0));           // [0, 0, 0, 0, 0]

// Create array of 5 elements and fill with 'x'
const filledArray = new Array(5).fill('x');
console.log(filledArray);               // ['x', 'x', 'x', 'x', 'x']

// Fill from index 1 to 4 with '*'
const partial = [1, 2, 3, 4, 5].fill('*', 1, 4);
console.log(partial);                   // [1, '*', '*', '*', 5]
```

⚠ The `fill()` method modifies the original array.

💡 **Knowledge Check:** If you call `fill()` with just a value parameter, what range of the array will be filled?

▶ Click to see answer

The `includes()` Method

The `includes()` method determines whether an array includes a certain value, returning `true` or `false`.

Syntax:

```
array.includes(element, start)
```

- `element` : The element to search for
- `start` : (Optional) The position to start searching from (default is 0)

Example:

```
const fruits = ['apple', 'banana', 'orange'];

console.log(fruits.includes('banana'));    // true
console.log(fruits.includes('grape'));    // false
console.log(fruits.includes('banana', 2)); // false (starts searching from index 2)
```

💡 **Knowledge Check:** What will `[1, 2, 3].includes('3')` return and why?

► Click to see answer

📌 The `indexOf()` and `lastIndexOf()` Methods

These methods return the first/last index at which a given element can be found in the array, or -1 if it's not present.

Syntax:

```
array.indexOf(element, start)
array.lastIndexOf(element, start)
```

Example:

```
const numbers = [10, 20, 30, 40, 30, 50];

console.log(numbers.indexOf(30));    // 2 (first occurrence)
console.log(numbers.lastIndexOf(30)); // 4 (last occurrence)
console.log(numbers.indexOf(60));    // -1 (not found)
console.log(numbers.indexOf(30, 3));  // 4 (starts searching from index 3)
```

💡 **Knowledge Check:** What does `indexOf()` return if the element doesn't exist in the array?

► Click to see answer

🔄 The `reverse()` Method

The `reverse()` method reverses the order of elements in an array and returns the modified array.

Syntax:

```
array.reverse()
```

Example:

```
const fruits = ['apple', 'banana', 'orange'];
fruits.reverse();
```

```
console.log(fruits); // ['orange', 'banana', 'apple']
```

⚠️ This method modifies the original array.

💡 **Knowledge Check:** How would you reverse an array without modifying the original array?

▶ Click to see answer

The sort() Method

The `sort()` method sorts the elements of an array in place and returns the sorted array.

1. The `sort()` method converts the element types into strings .
2. The default sorting order is ascending(alphabetical) order .

Syntax:

```
array.sort(compareFunction)
```

Example:

```
// Sorting strings (alphabetically)
const fruits = ['banana', 'apple', 'orange', 'grape'];
fruits.sort();
console.log(fruits); // ['apple', 'banana', 'grape', 'orange']

// Sorting numbers (by default, numbers are sorted as strings)
const numbers = [40, 100, 1, 5, 25, 10];
numbers.sort();
console.log(numbers); // [1, 10, 100, 25, 40, 5] (incorrect numerical order)

// To sort numbers correctly, use a compare function
numbers.sort((a, b) => a - b);
console.log(numbers); // [1, 5, 10, 25, 40, 100] (correct numerical order)

const artists = ["singer", "actor", "painter", "Musician"];
artists.sort(function(a, b){
    return a===b ? 0 : a > b ? -1 : 1 //comparator function descending order
})

//✅output: ['singer', 'painter', 'musician', 'actor']
```

⚠️ The default `sort()` converts elements to strings and compares their UTF-16 code units values.

💡 **Knowledge Check:** Why does `[10, 2, 5, 100].sort()` give `[10, 100, 2, 5]` instead of `[2, 5, 10, 100]` ?

▶ Click to see answer

✂ The splice() Method

The `splice()` method changes the contents of an array by removing or replacing existing elements and/or adding new elements in place.

Syntax:

```
array.splice(start, deleteCount, item1, item2, ...)
```

- `start` : Index at which to start changing the array
- `deleteCount` : (Optional) Number of elements to remove
- `item1, item2, ...` : (Optional) Elements to add at the start position

Example:

```
// Remove elements
const months = ['Jan', 'Feb', 'March', 'April', 'May'];
const removed = months.splice(1, 2);
console.log(months); // ['Jan', 'April', 'May']
console.log(removed); // ['Feb', 'March']

// Add elements without removing
const days = ['Mon', 'Tue', 'Thu', 'Fri'];
days.splice(2, 0, 'Wed');
console.log(days); // ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']

// Replace elements
const colors = ['red', 'green', 'purple'];
colors.splice(2, 1, 'blue');
console.log(colors); // ['red', 'green', 'blue']
```

⚠ This method modifies the original array.

💡 **Knowledge Check:** How would you use `splice()` to insert an element in the middle of an array without removing any elements?

▶ Click to see answer

📌 The at() Method

The `at()` method takes an integer value and returns the item at that index, allowing for positive and negative integers.

For positive integer it returns in normal default order from 0th index.

For positive integer it returns in reverse order meaning from last .

Syntax:

```
array.at(index)
```

Example:

```
const colors = ['red', 'green', 'blue', 'yellow'];

console.log(colors.at(2));    // 'blue'
console.log(colors.at(-1));   // 'yellow' (last element)
console.log(colors.at(-2));   // 'blue' (second last element)
```

The negative index counts from the end of the array, which is especially useful for accessing the last elements without knowing the array length.

 **Knowledge Check:** What's the difference between `arr[arr.length-1]` and `arr.at(-1)` ?

► Click to see answer

The `copyWithin()` Method

The `copyWithin()` method copies part of an array to another location in the same array and returns it without modifying its length.

Syntax:

```
array.copyWithin(target, start, end)
```

- **target** : Index position where copied elements should be placed
- **start** : (Optional) Index to start copying elements from (default is 0)
- **end** : (Optional) Index to stop copying elements from (default is array length)

Example:

```
const numbers = [1, 2, 3, 4, 5];

// Copy elements from index 3 to index 0
console.log(numbers.copyWithin(0, 3)); // [4, 5, 3, 4, 5]

// Copy elements from index 1 to index 3 at position 0
```

```
const letters = ['a', 'b', 'c', 'd', 'e'];
console.log(letters.copyWithin(0, 1, 3)); // ['b', 'c', 'c', 'd', 'e']
```

⚠️ This method modifies the original array.

💡 **Knowledge Check:** What happens if the `target` is a negative number?

▶ Click to see answer

The flat() Method

The `flat()` method creates a new array with all sub-array elements concatenated into it recursively up to the specified depth.

Syntax:

```
array.flat(depth)
```

- `depth` : (Optional) The depth level specifying how deep a nested array structure should be flattened (default is 1)

Example:

```
// Flatten one level
const nestedArray = [1, 2, [3, 4]];
console.log(nestedArray.flat()); // [1, 2, 3, 4]

// Flatten deeper nested arrays
const deepArray = [1, [2, [3, [4, 5]]]];
console.log(deepArray.flat());      // [1, 2, [3, [4, 5]]]
console.log(deepArray.flat(2));     // [1, 2, 3, [4, 5]]
console.log(deepArray.flat(3));     // [1, 2, 3, 4, 5]
console.log(deepArray.flat(Infinity)); // [1, 2, 3, 4, 5]
```

💡 **Knowledge Check:** How would you completely flatten an array with unknown nesting depth?

▶ Click to see answer

Grouping Elements in Array

JavaScript provides several ways to group array elements:

Object.groupBy() (ES2023)

The `Object.groupBy()` static method groups the elements of a given iterable according to the string values returned by a provided callback function.

Syntax:

```
Object.groupBy(items, callbackFn)
```

Example:

```
// Note: This is a newer feature, may need a modern environment
```

```
const inventory = [
  { name: 'apple', type: 'fruit' },
  { name: 'onion', type: 'vegetable' },
  { name: 'banana', type: 'fruit' },
  { name: 'carrot', type: 'vegetable' }
];

const groupedByType = Object.groupBy(inventory, item => item.type);
console.log(groupedByType);
/* Result:
{
  fruit: [
    { name: 'apple', type: 'fruit' },
    { name: 'banana', type: 'fruit' }
  ],
  vegetable: [
    { name: 'onion', type: 'vegetable' },
    { name: 'carrot', type: 'vegetable' }
  ]
}
*/
```

Example 2:

```
{
  const employees = [
    { name: "Bob", dept: "Engineering", salary: 5000 },
    { name: "Alex", dept: "HR", salary: 3000 },
    { name: "Ravi", dept: "Engineering", salary: 7000 },
    { name: "John", dept: "Engineering", salary: 1000 },
    { name: "Tom", dept: "Sales", salary: 6000 },
  ];

  const groupedByDet = Object.groupBy(employees, ({ dept }) => dept);
  console.log(groupedByDet);

  const groupedByMoreThan5000 = Object.groupBy(employees, ({ salary }) => {
    return salary >= 5000 ? "More than 5k" : "Less than 5k";
  });
  console.log(groupedByMoreThan5000);
}
```

```
/*  
run and see the output  
*/
```

Array.prototype.group() (Proposal)

There's also a proposed method for arrays directly:

```
// Note: This may not be available in all environments yet  
const result = inventory.group(item => item.type);
```

💡 **Note:** Since these grouping methods are newer features, you might need to use polyfills or alternative approaches in older environments.

💡 **Knowledge Check:** How would you group an array of numbers by whether they are odd or even?

▶ Click to see answer

Immutability

Immutability is a programming concept where data can't be changed after creation. In JavaScript, it's important for predictability, debugging, and performance optimization.

The toReversed() Method

The `toReversed()` method creates a new array with the elements in reversed order without modifying the original array.

Syntax:

```
array.toReversed()
```

Example:

```
const original = [1, 2, 3, 4, 5];  
const reversed = original.toReversed();  
  
console.log(reversed); // [5, 4, 3, 2, 1]  
console.log(original); // [1, 2, 3, 4, 5] (unchanged)
```

💡 **Knowledge Check:** What's the difference between `reverse()` and `toReversed()` ?

▶ Click to see answer

The toSorted() Method

The `toSorted()` method returns a new array with the elements sorted without modifying the original array.

Syntax:

```
array.toSorted(compareFunction)
```

Example:

```
const numbers = [3, 1, 4, 1, 5];
const sorted = numbers.toSorted();

console.log(sorted);    // [1, 1, 3, 4, 5]
console.log(numbers);   // [3, 1, 4, 1, 5] (unchanged)

// With custom compare function
const sortedDesc = numbers.toSorted((a, b) => b - a);
console.log(sortedDesc); // [5, 4, 3, 1, 1]
```

 **Knowledge Check:** When would you prefer `toSorted()` over `sort()` ?

► [Click to see answer](#)

The toSpliced() Method

The `toSpliced()` method returns a new array with some elements removed and/or replaced without modifying the original array.

Syntax:

```
array.toSpliced(start, deleteCount, item1, item2, ...)
```

Example:

```
const original = ['apple', 'banana', 'cherry', 'date'];

// Remove elements
const removed = original.toSpliced(1, 2);
console.log(removed);    // ['apple', 'date']
console.log(original);   // ['apple', 'banana', 'cherry', 'date'] (unchanged)

// Add elements without removing
const added = original.toSpliced(2, 0, 'kiwi', 'mango');
console.log(added);      // ['apple', 'banana', 'kiwi', 'mango', 'cherry', 'date']
```

```
// Replace elements
const replaced = original.toSpliced(1, 2, 'strawberry');
console.log(replaced); // ['apple', 'strawberry', 'date']
```

💡 **Knowledge Check:** How would you use `toSpliced()` to replace the middle element of an array with two new elements?

▶ Click to see answer

The with() Method

The `with()` method creates a new array with an element replaced at the specified index without modifying the original array.

Syntax:

```
array.with(index, value)
```

Example:

```
const colors = ['red', 'green', 'blue'];
const newColors = colors.with(1, 'yellow');

console.log(newColors); // ['red', 'yellow', 'blue']
console.log(colors);    // ['red', 'green', 'blue'] (unchanged)

// Using negative index
const updated = colors.with(-1, 'purple');
console.log(updated);   // ['red', 'green', 'purple']
```

💡 **Knowledge Check:** What happens if you use an index that's out of bounds with the `with()` method?

▶ Click to see answer

Practice Tasks

Basic Practice Tasks:

- Concatenation Challenge:** Create three arrays of your favorite foods, drinks, and desserts. Use the `concat()` method to combine them into a "menu" array.
- Custom Join:** Given the array `['JavaScript', 'is', 'awesome']`, use the `join()` method to create the sentence "JavaScript is awesome!" (with proper spacing and punctuation).

3. **Array Filling:** Create an array of length 10 and fill the first half with "A" and the second half with "B".
4. **Element Finder:** Given the array `const mixed = [42, "hello", true, null, "world"]`, write code to check if "hello" is included and find its index.
5. **Reversing Without Mutation:** Given an array `const numbers = [1, 2, 3, 4, 5]`, create a new reversed array without modifying the original using both the old way and the new `toReversed()` method.

Intermediate Practice Tasks:

6. **Sorting Challenge:** Create an array of objects representing people with name and age properties. Sort them by age in ascending order using both `sort()` and `toSorted()`.

```
const people = [  
  { name: "Alice", age: 25 },  
  { name: "Bob", age: 18 },  
  { name: "Charlie", age: 32 }  
];
```

7. **Splicing Practice:** Given the array `const weekdays = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']`:

- Use `splice()` to add 'Sat' and 'Sun' at the end
- Use `toSpliced()` to create a new array with 'Weekend' replacing 'Sat' and 'Sun'

8. **Flattening Nested Arrays:** Flatten the following array to a single level using the appropriate method:

```
const nested = [1, [2, 3], [4, [5, 6, [7, 8]]], 9];
```

9. **Grouping Exercise:** Given an array of numbers from 1 to 10, create two different groups: even and odd numbers.
10. **Immutable Replacement:** Given the array `const colors = ['red', 'green', 'blue', 'yellow', 'purple']`, use the `with()` method to create a new array where 'green' is replaced with 'emerald' and 'blue' is replaced with 'azure'.

Advanced Practice Tasks:

11. **Custom Array Transformation:** Create a function that takes an array of numbers and returns a new array where each element is transformed in the following way:
 - If the number is divisible by 3, multiply it by 2
 - If the number is divisible by 5, subtract 1

- If the number is divisible by both 3 and 5, replace it with 0
 - Otherwise, keep it as is Test your function with the array `[1, 3, 5, 15, 7, 9, 10]` .
12. **Multi-method Chain:** Start with the array `[5, 2, 8, 1, 9]` and chain together at least three different array methods to transform it into a specific result of your choosing. Explain each step.
13. **Implement Your Own Immutable Method:** Create your own version of an immutable array method (e.g., `myToFilter()`) that works like the original but doesn't modify the source array.

Key Takeaways

1. JavaScript provides numerous array methods that make working with arrays easier and more efficient.

2. Some array methods modify the original array (mutable methods):

- `reverse()` 
- `sort()` 
- `splice()` 
- `fill()` 
- `copyWithin()` 

3. Other methods return a new array without changing the original (immutable methods):

- `concat()` 
- `join()` (returns a string) 
- `includes()` (returns a boolean) 
- `indexOf()` / `lastIndexOf()` (returns a number) 
- `flat()` 
- `toReversed()` 
- `toSorted()` 
- `toSpliced()` 
- `with()` 

4. The newer immutable methods (`toReversed()` , `toSorted()` , `toSpliced()` , `with()`) provide safer alternatives to their mutable counterparts, helping to avoid unintended side effects.

5. Understanding when to use mutable vs. immutable methods is crucial for writing predictable and maintainable JavaScript code.

Mnemonic Table: "SCARF vs. JUSTICE"

Type	Method	What it Does	Mnemonic	Remember By
 Mutable	sort()	Orders elements	Sort	Sorts Change Arrays Really Fast (SCARF)
 Mutable	copyWithin()	Copies part of array to another location	CopyWithin	
 Mutable	fill()	Fills elements with static value	Alter (fill)	
 Mutable	reverse()	Reverses array order	Reverse	
 Mutable	splice()	Adds/removes elements	Fix (splice)	
 Immutable	join()	Creates string from array	Join	Just Use Safe Techniques In Code Everywhere (JUSTICE)
 Immutable	includes()	Checks if value exists	Uncover (includes)	
 Immutable	slice()	Returns portion of array	Slice	
 Immutable	toSorted()	Returns sorted array	ToSorted	
 Immutable	indexOf()	Finds element index	IndexOf	
 Immutable	concat()	Merges arrays	Concat	
 Immutable	at()	Gets element at index	Extract (at)	

JavaScript Arrays: Static Methods & Iterator Methods

Table of Contents

- [Static Array Methods](#)

- [Array-Like Objects](#)
- [Array.from\(\)](#)
- [Array.fromAsync\(\)](#)
- [Array.of\(\)](#)
- [Array Iterator Methods](#)
 - [filter\(\)](#)
 - [map\(\)](#)
 - [reduce\(\)](#)
 - [reduceRight\(\)](#)
 - [some\(\)](#)
 - [every\(\)](#)
 - [find\(\)](#)
 - [findIndex\(\)](#)
 - [findLast\(\)](#)
 - [findLastIndex\(\)](#)
 - [Array Method Chaining](#)
 - [forEach\(\)](#)
 - [entries\(\)](#)
 - [values\(\)](#)
 - [flatMap\(\)](#)
- [Practice Exercises](#)
- [Memory Aids](#)



Static Array Methods

Static array methods are called directly on the Array constructor (like `Array.from()`) rather than on array instances. These methods help with creating or manipulating arrays in different ways.

◆ Array-Like Objects

What are Array-Like Objects?

Array-like objects are objects that have:

-  Numeric indices (0, 1, 2, etc.)
-  A `length` property
-  But they don't have array methods like `push()` , `pop()` , etc.

Common examples of array-like objects:

-  DOM collections (like `HTMLCollection` , `NodeList`)

-  The arguments object in functions
-  String objects (as strings have indices and length)

```
// Example of an array-like object
const arrayLike = {
  0: "I",
  1: "am",
  2: "array-like",
  length: 3
};

// Access like an array
console.log(arrayLike[0]); // "I"
console.log(arrayLike.length); // 3

// But it's not truly an array
console.log(Array.isArray(arrayLike)); // false
console.log(arrayLike instanceof Object); // true

// The arguments object is array-like
function showArgs() {
  console.log(arguments); // Array-like object
  console.log(Array.isArray(arguments)); // false
}
```

 **Quick Check:** What makes an object "array-like" but not a true array?

► Solution

◆ Array.from()

The `Array.from()` method creates a new array from:

-  Array-like objects
-  Iterable objects (like Map, Set)

Syntax:

```
Array.from(arrayLike[, mapFn[, thisArg]])
```

Parameters:

- `arrayLike` : The object to convert to an array
- `mapFn` (optional): Function to call on each element
- `thisArg` (optional): Value to use as `this` in the `mapFn`

Examples:

```
// From array-like object
const arrayLike = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
const newArray = Array.from(arrayLike);
console.log(newArray); // ['a', 'b', 'c']

// With a map function
const mapped = Array.from(arrayLike, x => x.toUpperCase());
console.log(mapped); // ['A', 'B', 'C']

// From a string
const chars = Array.from('hello');
console.log(chars); // ['h', 'e', 'l', 'l', 'o']

// From DOM elements
const listItems = document.querySelectorAll('li');
const listArray = Array.from(listItems);
// Now you can use array methods like map, filter, etc.

// Create an array of numbers
const rangeArray = Array.from({length: 5}, (_, index) => index + 1);
console.log(rangeArray); // [1, 2, 3, 4, 5]
```

 **Quick Check:** How would you use `Array.from()` to create an array of five numbers, each multiplied by 2?

► Solution

◆ `Array.fromAsync()`

The `Array.fromAsync()` method creates a new array from asynchronous iterable objects, awaiting each promise.

Syntax:

```
Array.fromAsync(asyncIterable[, mapFn[, thisArg]])
```

Parameters:

- Same as `Array.from()`, but works with promises

Examples:

```
// From an array-like object of promises
const promises = {
  0: Promise.resolve('JavaScript'),
  1: Promise.resolve('TypeScript'),
  2: Promise.resolve('Python'),
  length: 3
}
```

```

};

// Process asynchronously
Array.fromAsync(promises)
  .then(result => console.log(result));
// Output: ['JavaScript', 'TypeScript', 'Python']

// From DOM elements that might need async processing
const imageCollection = document.getElementsByTagName('img');
Array.fromAsync(imageCollection)
  .then(images => {
    // Process images after all are loaded
  });

```

⚠ **Note:** `Array.fromAsync()` is a newer method and might not be supported in all browsers.

◆ `Array.of()`

The `Array.of()` method creates a new array from a variable number of arguments, regardless of type or number.

Syntax:

```
Array.of(element0[, element1[, ...[, elementN]]])
```

How it's different from the Array constructor:

- ❗ `new Array(3)` creates an array with length 3 (with empty slots)
- ✅ `Array.of(3)` creates an array with a single element: `[3]`

Examples:

```

// Create arrays with various elements
const array1 = Array.of(7);
console.log(array1); // [7]

const array2 = Array.of(1, 2, 3);
console.log(array2); // [1, 2, 3]

// Mixing data types is no problem
const array3 = Array.of(true, 'hello', {name: 'John'}, [1, 2]);
console.log(array3); // [true, 'hello', {name: 'John'}, [1, 2]]

// Compare with regular Array constructor
const withConstructor = new Array(3); // Creates array with 3 empty slots
console.log(withConstructor); // [empty x 3]
console.log(withConstructor.length); // 3

const withOf = Array.of(3); // Creates array with single element: 3

```

```
console.log(withOf); // [3]
console.log(withOf.length); // 1
```

 **Quick Check:** What's the difference between `new Array(5)` and `Array.of(5)` ?

► Solution

Array Iterator Methods

Array iterator methods process each item in an array and typically return a new value, array, or perform an action on each element.

◆ `filter()`

The `filter()` method creates a new array with elements that pass a test implemented by a provided function.

Syntax:

```
const newArray = array.filter(callback(element, index, array) {
  // Return true to keep the element, false to exclude it
});
```

Parameters:

- `callback` : Function to test each element
 - `element` : Current element being processed
 - `index` (optional): Index of the current element
 - `array` (optional): The array `filter()` was called upon

It is an array method which takes a callback function which works like a test function that runs on every elements of the array, if the test function returns `true` then the element on which we are running the test function will be part of the returned array else not.

Examples:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// Get only even numbers
const evenNumbers = numbers.filter(num => num % 2 === 0);
console.log(evenNumbers); // [2, 4, 6, 8, 10]

// More complex filtering
const customers = [
  { name: 'John', age: 25, active: true },
  { name: 'Jane', age: 17, active: false },
```

```

    { name: 'Bob', age: 35, active: true },
    { name: 'Mary', age: 14, active: true }
  ];

  // Get active adult customers (age >= 18)
  const activeAdults = customers.filter(customer =>
    customer.age >= 18 && customer.active
  );
  console.log(activeAdults);
  // Output: [{ name: 'John', age: 25, active: true }, { name: 'Bob', age: 35, active: true }]

```

 **Quick Check:** How would you filter an array of numbers to get only those greater than 10 and less than 20?

► Solution

◆ map()

The `map()` method creates a new array with the results of calling a function on every element in the calling array.

Syntax:

```

const newArray = array.map(callback(element, index, array) {
  // Return the transformed element
});

```

Parameters:

- Same as for `filter()`

Examples:

```

const numbers = [1, 2, 3, 4];

// Multiply each number by 2
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6, 8]

// Transform objects
const users = [
  { firstName: 'John', lastName: 'Doe' },
  { firstName: 'Jane', lastName: 'Smith' }
];

const fullNames = users.map(user =>
  `${user.firstName} ${user.lastName}`
);
console.log(fullNames); // ['John Doe', 'Jane Smith']

```



```
// Example with index parameter
const indexed = numbers.map((num, index) =>
  `Item ${index}: ${num}`
);
console.log(indexed); // ['Item 0: 1', 'Item 1: 2', 'Item 2: 3', 'Item 3: 4']
```

 **Quick Check:** How would you use `map()` to convert an array of prices to include a 10% discount?

► Solution

◆ `reduce()`

The `reduce()` method executes a reducer function on each element of the array, resulting in a single output value.

Syntax:

```
const result = array.reduce(callback(accumulator, currentValue, index, array) {
  // Return the updated accumulator
}, initialValue);
```

Parameters:

- `callback` : Function to execute on each element
 - `accumulator` : Accumulated value returned after each callback
 - `currentValue` : Current element being processed
 - `index` (optional): Index of the current element
 - `array` (optional): The array `reduce()` was called upon
- `initialValue` (optional): Value to use as first argument in first call

Examples:

```
const numbers = [1, 2, 3, 4, 5];

// Sum all numbers
const sum = numbers.reduce((accumulator, current) =>
  accumulator + current, 0);
console.log(sum); // 15

// Without initial value (uses first element as initial value)
const sumNoInitial = numbers.reduce((accumulator, current) =>
  accumulator + current);
console.log(sumNoInitial); // 15

// Finding the maximum value
```

```

const max = numbers.reduce((max, current) =>
  Math.max(max, current), -Infinity);
console.log(max); // 5

// Counting occurrences in an array
const fruits = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple'];
const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});
console.log(count); // { apple: 3, banana: 2, orange: 1 }

```

 **Quick Check:** How would you use `reduce()` to multiply all numbers in an array together?

► Solution

◆ `reduceRight()`

The `reduceRight()` method is like `reduce()`, but it processes the array from right to left (from the last element to the first).

Syntax:

```

const result = array.reduceRight(callback(accumulator, currentValue, index, array) {
  // Return the updated accumulator
}, initialValue);

```

Parameters:

- Same as `reduce()`

Examples:

```

const numbers = [1, 2, 3, 4, 5];

// Sum all numbers (right to left)
const sum = numbers.reduceRight((accumulator, current) =>
  accumulator + current, 0);
console.log(sum); // 15 (same as normal reduce in this case)

// Where order matters:
const values = [10, 5, 2];

// Left to right: (10 - 5) - 2 = 3
const subtractLeft = values.reduce((acc, val) => acc - val);
console.log(subtractLeft); // 3

// Right to left: 2 - (5 - 10) = 7
const subtractRight = values.reduceRight((acc, val) => acc - val);

```

```
console.log(subtractRight); // -7

// Flattening nested arrays
const nestedArrays = [[0, 1], [2, 3], [4, 5]];
const flattened = nestedArrays.reduceRight((acc, curr) =>
  acc.concat(curr), []);
console.log(flattened); // [4, 5, 2, 3, 0, 1]
```

 **Quick Check:** What kind of operations would benefit from using `reduceRight()` instead of `reduce()` ?

► Solution

◆ `some()`

The `some()` method tests whether at least one element in the array passes the test implemented by the provided function.

Syntax:

```
const result = array.some(callback(element, index, array) {
  // Return true to indicate test passed, false otherwise
});
```

Parameters:

- Same as for `filter()`

Examples:

```
const numbers = [1, 2, 3, 4, 5];

// Check if at least one number is even
const hasEven = numbers.some(num => num % 2 === 0);
console.log(hasEven); // true

// Check if at least one number is greater than 10
const hasLarge = numbers.some(num => num > 10);
console.log(hasLarge); // false

const team = [
  { name: 'John', age: 22, active: true },
  { name: 'Jane', age: 17, active: false },
  { name: 'Bob', age: 35, active: true }
];

// Check if any team member is under 18
```

```
const hasMinor = team.some(member => member.age < 18);
console.log(hasMinor); // true
```

 **Quick Check:** How would you check if an array of strings contains at least one email address (containing '@')?

► Solution

◆ every()

The `every()` method tests whether all elements in the array pass the test implemented by the provided function.

Syntax:

```
const result = array.every(callback(element, index, array) {
  // Return true if the element passes the test, false otherwise
});
```

Parameters:

- Same as for `filter()`

Examples:

```
const numbers = [2, 4, 6, 8, 10];

// Check if ALL numbers are even
const allEven = numbers.every(num => num % 2 === 0);
console.log(allEven); // true

// Check if ALL numbers are greater than 5
const allLarge = numbers.every(num => num > 5);
console.log(allLarge); // false

const students = [
  { name: 'John', score: 85 },
  { name: 'Jane', score: 92 },
  { name: 'Bob', score: 78 }
];

// Check if all students passed (score >= 70)
const allPassed = students.every(student => student.score >= 70);
console.log(allPassed); // true
```

 **Quick Check:** Given an array of user objects, how would you check if all users are adults (age ≥ 18)?

► Solution

◆ find()

The `find()` method returns the first element in the array that satisfies the provided testing function.

Syntax:

```
const result = array.find(callback(element, index, array) {  
  // Return true for the element to find  
});
```

Parameters:

- Same as for `filter()`

Examples:

```
const numbers = [5, 12, 8, 130, 44];  
  
// Find the first number greater than 10  
const found = numbers.find(num => num > 10);  
console.log(found); // 12  
  
// Find a specific user  
const users = [  
  { id: 1, name: 'John' },  
  { id: 2, name: 'Jane' },  
  { id: 3, name: 'Bob' }  
];  
  
const user = users.find(user => user.id === 2);  
console.log(user); // { id: 2, name: 'Jane' }  
  
// Returns undefined if no element found  
const missingUser = users.find(user => user.id === 99);  
console.log(missingUser); // undefined
```

🧠 **Quick Check:** How would you find the first product in an array that costs less than \$50?

► Solution

◆ findIndex()

The `findIndex()` method returns the index of the first element in the array that satisfies the provided testing function.

Syntax:

```
const index = array.findIndex(callback(element, index, array) {  
  // Return true for the element to find  
});
```

Parameters:

- Same as for `filter()`

Examples:

```
const numbers = [5, 12, 8, 130, 44];  
  
// Find the index of the first number greater than 10  
const foundIndex = numbers.findIndex(num => num > 10);  
console.log(foundIndex); // 1 (index of 12)  
  
// Find index of a specific user  
const users = [  
  { id: 1, name: 'John' },  
  { id: 2, name: 'Jane' },  
  { id: 3, name: 'Bob' }  
];  
  
const userIndex = users.findIndex(user => user.id === 3);  
console.log(userIndex); // 2 (index of Bob)  
  
// Returns -1 if no element found  
const missingIndex = users.findIndex(user => user.id === 99);  
console.log(missingIndex); // -1
```

 **Quick Check:** How would you find the index of the first negative number in an array?

► Solution

◆ `findLast()`

The `findLast()` method returns the last element in the array that satisfies the provided testing function.

Syntax:

```
const result = array.findLast(callback(element, index, array) {  
  // Return true for the element to find  
});
```

Parameters:

- Same as for `filter()`

Examples:

```
const numbers = [5, 12, 8, 130, 44];

// Find the last number less than 50
const lastSmall = numbers.findLast(num => num < 50);
console.log(lastSmall); // 44

// Find the last even number
const lastEven = numbers.findLast(num => num % 2 === 0);
console.log(lastEven); // 44

const users = [
  { role: 'admin', name: 'John' },
  { role: 'user', name: 'Jane' },
  { role: 'admin', name: 'Bob' }
];

// Find the last admin user
const lastAdmin = users.findLast(user => user.role === 'admin');
console.log(lastAdmin); // { role: 'admin', name: 'Bob' }
```

 **Quick Check:** How would you find the last item in an inventory array that is in stock?

► Solution

◆ `findLastIndex()`

The `findLastIndex()` method returns the index of the last element in the array that satisfies the provided testing function.

Syntax:

```
const index = array.findLastIndex(callback(element, index, array) {
  // Return true for the element to find
});
```

Parameters:

- Same as for `filter()`

Examples:

```
const numbers = [5, 12, 8, 130, 44];

// Find the index of the last number less than 50
```

```
const lastSmallIndex = numbers.findLastIndex(num => num < 50);
console.log(lastSmallIndex); // 4 (index of 44)

const fruits = ['apple', 'banana', 'orange', 'apple', 'grape'];

// Find the index of the last occurrence of 'apple'
const lastAppleIndex = fruits.findLastIndex(fruit => fruit === 'apple');
console.log(lastAppleIndex); // 3
```

 **Quick Check:** Given an array of test scores, how would you find the index of the last failing score (below 60)?

► Solution

◆ Array Method Chaining

Method chaining allows you to perform multiple array operations in sequence, where each method works on the result of the previous method.

Examples:

```
const products = [
  { id: 1, name: 'Laptop', price: 1200, inStock: true },
  { id: 2, name: 'Phone', price: 750, inStock: true },
  { id: 3, name: 'Tablet', price: 450, inStock: false },
  { id: 4, name: 'Headphones', price: 50, inStock: true }
];

// Get the total price of all in-stock products
const totalInStockValue = products
  .filter(product => product.inStock) // First: keep only in-stock items
  .map(product => product.price) // Then: extract just the price
  .reduce((total, price) => total + price, 0); // Finally: sum up all prices

console.log(totalInStockValue); // 2000 (1200 + 750 + 50)

// Find names of expensive in-stock products
const expensiveProductNames = products
  .filter(product => product.price > 500 && product.inStock)
  .map(product => product.name);

console.log(expensiveProductNames); // ['Laptop', 'Phone']
```

Benefits of method chaining:

- ✨ Cleaner, more readable code
- 🚫 Avoids creating unnecessary intermediate variables
- 🔗 Creates a clear data transformation pipeline

 **Quick Check:** How would you chain methods to find the sum of squares of even numbers in an array?

► Solution

◆ `forEach()`

The `forEach()` method executes a provided function once for each array element.

Syntax:

```
array.forEach(callback(element, index, array) {  
  // Execute code for each element  
});
```

Parameters:

- Same as for `filter()`

Examples:

```
const numbers = [1, 2, 3, 4, 5];  
  
// Print each number  
numbers.forEach(num => {  
  console.log(num);  
});  
// Output: 1, 2, 3, 4, 5 (each on own line)  
  
// Using index parameter  
numbers.forEach((num, index) => {  
  console.log(`Element at index ${index} is ${num}`);  
});  
  
// Sum using forEach  
let sum = 0;  
numbers.forEach(num => {  
  sum += num;  
});  
console.log(sum); // 15
```

Key differences from other iterator methods:

-  `forEach()` always returns `undefined`
-  You cannot break out of a `forEach()` loop early (unlike a regular `for` loop)
-  It's meant for performing side effects, not transforming data

 **Quick Check:** How would you use `forEach()` to calculate both the sum and product of all numbers in an array?

► Solution

◆ `entries()`

The `entries()` method returns a new Array Iterator object that contains key/value pairs for each index in the array.

Syntax:

```
const iterator = array.entries();
```

Examples:

```
const fruits = ['apple', 'banana', 'cherry'];
const iterator = fruits.entries();

// Manually accessing the iterator
console.log(iterator.next().value); // [0, 'apple']
console.log(iterator.next().value); // [1, 'banana']

// Using a for...of loop (more common)
const colors = ['red', 'green', 'blue'];
for (const [index, color] of colors.entries()) {
  console.log(`Color at position ${index} is ${color}`);
}
// Output:
// "Color at position 0 is red"
// "Color at position 1 is green"
// "Color at position 2 is blue"

// Converting the iterator to an array
const months = ['Jan', 'Feb', 'Mar'];
const entriesArray = [...months.entries()];
console.log(entriesArray);
// Output: [[0, 'Jan'], [1, 'Feb'], [2, 'Mar']]
```

 **Quick Check:** How could you use `entries()` to create an object mapping array values to their indices?

◆ `values()`

The `values()` method returns a new Array Iterator object that contains the values for each index in the array.

Syntax:

```
const iterator = array.values();
```

Examples:

```
const fruits = ['apple', 'banana', 'cherry'];
const iterator = fruits.values();

// Manually accessing the iterator
console.log(iterator.next().value); // 'apple'
console.log(iterator.next().value); // 'banana'

// Using a for...of loop (more common)
const colors = ['red', 'green', 'blue'];
for (const color of colors.values()) {
  console.log(color);
}
// Output:
// "red"
// "green"
// "blue"

// Converting the iterator to an array (essentially clones the array)
const months = ['Jan', 'Feb', 'Mar'];
const valuesArray = [...months.values()];
console.log(valuesArray); // ['Jan', 'Feb', 'Mar']
```

💡 **Quick Check:** When might you use `values()` instead of directly iterating over the array?

◆ flatMap()

The `flatMap()` method first maps each element using a mapping function, then flattens the result into a new array.

Syntax:

```
const newArray = array.flatMap(callback(currentValue, index, array) {
  // Return an array or a single value to be flattened
});
```

Parameters:

- Same as for `map()`

Examples:

```
const numbers = [1, 2, 3, 4];
```

```
// Using map() vs flatMap()
const mapped = numbers.map(x => [x * 2]);
console.log(mapped); // [[2], [4], [6], [8]]

const flatMapped = numbers.flatMap(x => [x * 2]);
console.log(flatMapped); // [2, 4, 6, 8]

// More complex example: Generate pairs
const pairs = numbers.flatMap(n => [n, n + 1]);
console.log(pairs); // [1, 2, 2, 3, 3, 4, 4, 5]

// Filter and map in one operation
const sentences = ["Hello world", "I am learning JavaScript"];
const words = sentences.flatMap(sentence => sentence.split(' '));
console.log(words); // ['Hello', 'world', 'I', 'am', 'learning', 'JavaScript']

// Can also be used to filter out items by returning empty arrays
const mixed = [1, 2, null, 4, undefined, 6];
const filtered = mixed.flatMap(item =>
  item != null ? [item] : []
);
console.log(filtered); // [1, 2, 4, 6]
```

 **Quick Check:** How would you use `flatMap()` to create an array of characters from an array of words?



Practice Exercises

Test your understanding of JavaScript array methods with these practice exercises:

Exercise 1: Static Methods

Create an array from the letters of the word "JavaScript" using a static array method.

► Solution

Exercise 2: Working with Array-Like Objects

Convert the following array-like object to a proper array and double each value:

```
const arrayLike = {
  0: 10,
  1: 20,
  2: 30,
  length: 3
};
```

► Solution

Exercise 3: Filter and Map

Given an array of products:

```
const products = [
  { id: 1, name: 'Laptop', price: 1200, category: 'Electronics' },
  { id: 2, name: 'Book', price: 20, category: 'Books' },
  { id: 3, name: 'Phone', price: 500, category: 'Electronics' },
  { id: 4, name: 'Desk', price: 350, category: 'Furniture' },
  { id: 5, name: 'Keyboard', price: 100, category: 'Electronics' }
];
```

Find all electronics products and return an array containing just their names in uppercase.

► Solution

Exercise 4: Reduce

Calculate the total price of all products in the previous array.

► Solution

Exercise 5: Find and Some

1. Find the first product that costs more than \$1000.
2. Check if there are any furniture products available.

► Solution

Exercise 6: Array Method Chaining

Calculate the average price of electronics products.

► Solution

Exercise 7: FlatMap

Convert an array of sentences into an array of unique words (no duplicates):

```
const sentences = [
  "JavaScript is amazing",
  "I love JavaScript",
  "Arrays are powerful in JavaScript"
];
```

► Solution

Exercise 8: ForEach vs Map

What's the difference between these two code snippets and what will they output?

```
// Snippet 1
const numbers1 = [1, 2, 3, 4];
const result1 = numbers1.forEach(num => num * 2);

// Snippet 2
const numbers2 = [1, 2, 3, 4];
const result2 = numbers2.map(num => num * 2);

console.log(result1);
console.log(result2);
```

► Solution

Exercise 9: Combining Methods

Given the following data:

```
const students = [
  { id: 1, name: "Alice", scores: [85, 90, 92] },
  { id: 2, name: "Bob", scores: [75, 80, 85] },
  { id: 3, name: "Charlie", scores: [90, 95, 85] },
  { id: 4, name: "Diana", scores: [65, 70, 75] }
];
```

Create an array containing objects with student names and their average scores, but only for students with an average score above 80.

► Solution

Exercise 10: Using Array.of() and Array.from()

Create a function that takes any number of arguments and returns an array of their squares.

► Solution

Summary: Static and Iterator Array Methods

Static Array Methods

- **Array-like objects:** Objects with numeric indices and a length property, but not true arrays
- **Array.from():** Creates a new array from array-like or iterable objects
- **Array.fromAsync():** Creates a new array from asynchronous iterables and promises

- **Array.of()**: Creates a new array from individual arguments

Iterator Methods

- **filter()**: Creates a new array with elements that pass a test
- **map()**: Creates a new array by transforming each element
- **reduce()**: Reduces the array to a single value (left to right)
- **reduceRight()**: Reduces the array to a single value (right to left)
- **some()**: Checks if at least one element passes a test
- **every()**: Checks if all elements pass a test
- **find()**: Returns the first element that passes a test
- **findIndex()**: Returns the index of the first element that passes a test
- **findLast()**: Returns the last element that passes a test
- **findLastIndex()**: Returns the index of the last element that passes a test
- **forEach()**: Executes a function for each element (no return value)
- **entries()**: Returns an iterator with key/value pairs [index, element]
- **values()**: Returns an iterator with array values
- **flatMap()**: Maps each element and flattens the result into a new array



JavaScript Array Methods Cheatsheet



Static Methods (Called on Array constructor)

Method	Purpose	When to Use	Memory Aid
 <code>Array.from()</code>	Creates array from array-like/iterable objects	Converting NodeLists, arguments, strings to arrays	" FROM other things"
 <code>Array.fromAsync()</code>	Creates array from async iterables	Working with promises or async data sources	" FROM promises"
 <code>Array.of()</code>	Creates array from individual arguments	When you need an array with specific elements	" OF these exact items"



Iterator Methods (Called on array instances)



Filtering & Finding

Method	Purpose	Return Value	When to Use	Memory Aid
 <code>filter()</code>	Keeps elements that pass a test	New array	Removing unwanted elements	" FILTER keeps what fits"
 <code>find()</code>	Returns first matching element	Element or undefined	Finding a specific item	" FIND first match"
 <code>findIndex()</code>	Returns index of first match	Number (index)	Finding position of an item	" FIND position"
 <code>findLast()</code>	Returns last matching element	Element or undefined	Finding last occurrence	" FIND last match"
 <code>findLastIndex()</code>	Returns index of last match	Number (index)	Finding position of last occurrence	" FIND last position"

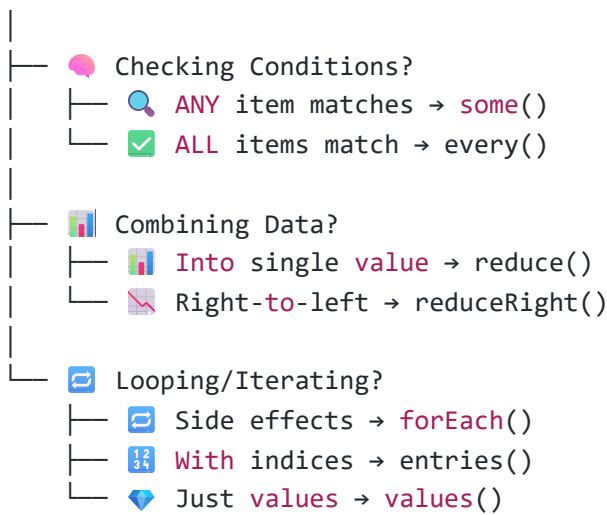
Testing & Checking

Method	Purpose	Return Value	When to Use	Memory Aid
 <code>some()</code>	Tests if ANY element passes	Boolean	Checking for at least one match	"Are there SOME ?"
 <code>every()</code>	Tests if ALL elements pass	Boolean	Validating all items	"Do EVERY one qualify?"

Transforming

Method	Purpose	Return Value	When to Use	Memory Aid
 <code>map()</code>	Transforms each element	New array	Converting data format	" MAP transforms each item"
 <code>flatMap()</code>	Maps then flattens results	New array	Transform and expand/contract items	" MAP then FLAT ten"

Reducing & Combining



Memory Techniques

FIRM-SEF Framework

- Find-Filter: For searching & filtering 
- Iterate: For looping (forEach, entries, values) 
- Reduce: For combining values 
- Map: For transforming data 
- Some-Every: For testing conditions 
- From-oF: For creating arrays 

"Return Value" Mnemonic: "BUNEIS"

- Boolean: some, every ✓ 
- Undefined: forEach 
- Number (index): findIndex, findLastIndex 
- Element: find, findLast 
- Iterator: values, entries 
- Same structure (new array): map, filter, flatMap 

Method Pairing

Remember methods in logical pairs:

-  some/every (any vs. all)
-  find/findIndex (item vs. position)
-  findLast/findLastIndex (last item vs. last position)
-  reduce/reduceRight (left-to-right vs. right-to-left)
-  Array.from/Array.of (from existing vs. from arguments)

💡 Visual Association Tips

- 🔪 `filter()` → Picture a sieve letting only certain **items** through
- 🔄 `map()` → Imagine a machine transforming **each item on** a conveyor belt
- 📊 `reduce()` → Visualize many **items** being combined **into one** container
- 🔍 `find()` → Think **of a** magnifying glass finding **the first** match
- ✅ `every()` → Picture checking every box **on** a checklist

🔄 Method Chaining Example

```
// Chain methods to find expensive products' names
products
  .filter(p => p.price > 100) // 🔪 Keep expensive items
  .map(p => p.name)           // 🔄 Extract just the names
  .sort()                     // 📄 Sort alphabetically
```

🎯 Quick Reference by Task

I want to...	Use this method
📊 Calculate a total	<code>reduce()</code>
🔪 Remove unwanted items	<code>filter()</code>
🔍 Find a specific item	<code>find()</code>
🔄 Transform all items	<code>map()</code>
✅ Check if all items match	<code>every()</code>
🔍 Check if any item matches	<code>some()</code>
🔄 Perform side effects	<code>forEach()</code>

These methods make JavaScript arrays incredibly powerful for data manipulation, transformation, and analysis. Understanding and mastering them is crucial for effective JavaScript programming.

📅 Next: Day - 16: Javascript Debugging with DEV tools and VS Code 📅

🚀 Happy JavaScript coding! 🚀

✅🚀 For solution of knowledge check and practise task visit github repo and telegram channel the links are given below 🙌

🚀 *Let's Connect & Level Up Together*

Follow me for daily JavaScript tips, insightful notes, and project-based learning.

🔗 X (Twitter): @_19_neeraj

💼 LinkedIn: [linkedin.com/in/neeraj-kumar1904](https://www.linkedin.com/in/neeraj-kumar1904)

💬 Telegram : [Neeraj.Dev](#)

🔍 Dive into the full notes on GitHub → **40 Days JavaScript:**
github.com/Neeraj05042001

© 2025 • Crafted with ❤️ by Neeraj

100 Remember: Practice is key to mastering JavaScript variables and data types!

100